

FACULTAD DE INGENIERÍA
UNIVERSIDAD NACIONAL DE ENTRE RÍOS
BIOINGENIERÍA



Tp N°2: Aplicaciones de estructuras jerárquicas y grafos

INTEGRANTE/S:

Muller González Brian Ezequiel,

Velazco Francisco,

Yang Luis Fernando

FECHA DE ENTREGA: 08 de Junio del 2026

LINK DEL REPOSITORIO:

<https://github.com/Brianmuller26/AyED2026c1-Muller-Velazco-Yang.git>

INTRODUCCIÓN

En el presente Trabajo Práctico N° 2 se aplican los conceptos teóricos desarrollados durante la cursada relacionados con estructuras jerárquicas y grafos, mediante la resolución de distintos problemas que requieren el uso de Tipos Abstractos de Datos (TADs) y algoritmos específicos para su implementación.

A lo largo del trabajo se desarrollan soluciones basadas en montículos binarios, colas de prioridad, árboles binarios de búsqueda balanceados (AVL) y grafos, analizando su comportamiento y eficiencia en diferentes situaciones. Para cada problema se seleccionó la estructura de datos más adecuada, considerando tanto la organización de la información como el costo computacional de las operaciones realizadas.

Asimismo, se realizó el análisis de complejidad temporal mediante notación Big-O de los algoritmos implementados, justificando las decisiones adoptadas en cada caso. También se empleó manejo de excepciones para la validación de datos y operaciones, además de la definición de precondiciones y postcondiciones para documentar el comportamiento esperado de los métodos desarrollados.

DESARROLLO DE LOS PROBLEMAS

Problema 1

Análisis y planteo

El problema consiste en administrar una sala de emergencias donde los pacientes deben ser atendidos según la gravedad de su estado clínico y no únicamente según el orden de llegada. Por lo tanto, la operación crítica del problema es la selección del próximo paciente a atender, ya que en cada instante debe obtenerse aquel que posea el mayor nivel de prioridad.

Los niveles de riesgo se representan mediante valores numéricos, donde 1 corresponde a riesgo crítico, 2 a riesgo moderado y 3 a riesgo bajo. En base a esto, nos resulta conveniente utilizar una estructura que permita acceder rápidamente al elemento de menor valor de prioridad. Por esta razón decidimos usar un Montículo Binario de mínimos. Esta estructura nos permite insertar pacientes a medida que ingresan al sistema y recuperar eficientemente el paciente con mayor prioridad clínica. Las operaciones de inserción y eliminación poseen complejidad temporal $O(\log n)$, mientras que la consulta del elemento prioritario se realiza en $O(1)$.

La elección de esta estructura resulta adecuada para escenarios donde los pacientes ingresan continuamente y el sistema debe responder rápidamente ante cambios en la cola de espera, manteniendo siempre disponible el paciente de mayor urgencia para ser atendido.

Implementación / Resolución

Implementamos una estructura de tipo Montículo Binario de mínimos mediante una lista que almacena los elementos. La posición de cada nodo dentro de la lista permite determinar sus hijos y su padre mediante operaciones aritméticas sobre los índices. El montículo mantiene la propiedad de orden utilizando los métodos de filtrado hacia arriba (`infiltrArriba`) y filtrado hacia abajo (`infiltrAbajo`).

Los pacientes son generados aleatoriamente durante la simulación y poseen un nivel de riesgo asociado. Cada nuevo paciente se inserta en el montículo mediante el método `insertar()`. Durante la inserción se compara el nivel de riesgo del paciente con el de sus ancestros.

Cuando corresponde atender a un paciente, utilizamos el método `eliminarMin()`, que extrae el elemento ubicado en la raíz del montículo. Dado que la raíz contiene siempre el paciente con menor valor de riesgo, con esto garantizamos que los pacientes críticos sean atendidos

antes que los moderados y estos antes que los de riesgo bajo. La comparación entre pacientes se realiza utilizando el método `get_riesgo()`.

Resultados y pruebas

Las pruebas que realizamos consisten en ejecutar la simulación de la sala de emergencias generando pacientes con distintos niveles de riesgo. Durante la ejecución verificamos que los pacientes atendidos correspondieran siempre al menor valor de riesgo disponible en la cola de espera.

Los resultados obtenidos mostraron que los pacientes críticos fueron seleccionados antes que los moderados y los de riesgo bajo, independientemente del orden de llegada, cumpliendo con lo que pedía el ejercicio.

Además, comprobamos que la inserción continua de nuevos pacientes y la eliminación de pacientes atendidos no afectan la consistencia de la estructura, preservándose la propiedad del montículo en todo momento.

Problema 2

Análisis y planteo

El problema consiste en desarrollar una base de datos de temperaturas que permita almacenar mediciones asociadas a fechas y realizar consultas eficientes sobre ellas. Entre las operaciones requeridas se encuentran la inserción de nuevas mediciones, la búsqueda de temperaturas para una fecha determinada, la eliminación de registros y la realización de consultas sobre rangos de fechas.

La operación crítica del sistema es la búsqueda y recuperación de información utilizando fechas como criterio de ordenamiento. Además, el conjunto de datos crece progresivamente a medida que se incorporan nuevas mediciones, por lo que resulta necesario mantener una estructura equilibrada que garantice tiempos de ejecución eficientes.

Para resolver el problema utilizamos un Árbol AVL. Esta estructura mantiene automáticamente el equilibrio mediante rotaciones cuando se producen desbalances, garantizando una altura logarítmica. Gracias a esto, las operaciones de inserción, búsqueda y eliminación poseen complejidad temporal $O(\log n)$ en el peor caso.

Las fechas ingresadas por el usuario las convertimos internamente a objetos `datetime`, pudiendo así compararlas cronológicamente y utilizarlas como claves dentro del árbol. De esta forma, las mediciones quedan ordenadas temporalmente y las consultas por rango pueden realizarse de manera eficiente mediante recorridos controlados del árbol.

Implementación / Resolución

Implementamos la solución mediante una clase denominada Temperaturas_DB, que encapsula un Árbol AVL utilizado para almacenar las mediciones de temperatura.

Cada registro se guarda utilizando como clave la fecha de medición y como valor la temperatura correspondiente. Antes de ser almacenadas, las fechas ingresadas como cadenas de caracteres en formato "dd/mm/aaaa" la convertimos a objetos datetime para facilitar las comparaciones cronológicas.

Las operaciones guardar_temperatura(), devolver_temperatura() y borrar_temperatura() utilizan directamente los métodos de inserción, búsqueda y eliminación provistos por el árbol AVL.

Para las consultas sobre intervalos de fechas, lo que hacemos es recorrer únicamente las ramas que pueden contener fechas comprendidas dentro del rango solicitado, obteniendo las mediciones ordenadas cronológicamente. A partir de los valores recuperados se calculan las temperaturas máximas y mínimas requeridas por la consigna.

Asimismo, incorporamos la lectura de un archivo de muestras que permite cargar automáticamente mediciones en la base de datos para facilitar las pruebas y validaciones del sistema.

Resultados y pruebas

A continuación se muestra la salida de datos de cada método:

```
La temperatura minima y maxima es: (11.9, 39.9)
La fecha de la cual se elimina la temperatura medida es: 07/01/2025
Las temperaturas medidas en las fechas es:
02/01/2025: 38.6 °C
03/01/2025: 25.8 °C
04/01/2025: 23.7 °C
05/01/2025: 18.6 °C
La cantidad de muestras de la DB es: 119
```

Las pruebas realizadas permitieron verificar el correcto funcionamiento de todas las operaciones definidas para la base de datos. También comprobamos que las temperaturas pueden almacenarse, recuperarse y eliminarse correctamente utilizando fechas como clave de búsqueda.

En las consultas por rango obtuvimos los resultados esperados, tanto para la obtención de temperaturas extremas como para la generación de listados ordenados cronológicamente. Además, la carga de datos desde archivo permitió validar el comportamiento de la estructura ante un conjunto significativo de mediciones.

El uso de un Árbol AVL nos garantiza que las operaciones fundamentales mantengan tiempos de ejecución eficientes aun cuando la cantidad de registros almacenados aumenta considerablemente.

Finalmente se nos pide mostrar en una tabla la complejidad Big-O para cada método implementado y hacer un breve análisis. Dicha tabla y análisis se presenta a continuación:

MÉTODO	COMPLEJIDAD
guardar_temperatura()	$O(\log n)$
devolver_temperatura()	$O(\log n)$
borrar_temperatura()	$O(\log n)$
cantidad_muestras()	$O(1)$
devolver_temperaturas()	$O(\log n + k)$
max_temp_rango()	$O(\log n + k)$
min_temp_rango()	$O(\log n + k)$
temp_extremos_rango()	$O(\log n + k)$

donde n es la cantidad total de mediciones almacenadas y k se refiere a la cantidad de mediciones encontradas dentro del rango consultado.

Las operaciones de inserción, búsqueda y eliminación presentan complejidad $O(\log n)$ debido a que el árbol AVL mantiene una altura logarítmica mediante rotaciones de balanceo. Las consultas sobre rangos requieren localizar el inicio del intervalo y recorrer únicamente los nodos comprendidos dentro de dicho rango. Por este motivo su complejidad puede expresarse como $O(\log n + k)$, donde k representa la cantidad de mediciones recuperadas. Finalmente, la operación `cantidad_muestras()` posee complejidad constante $O(1)$ ya que únicamente devuelve el tamaño almacenado internamente por la estructura.

Algo que también realizamos son pruebas de inserción destinadas a generar desbalances en el árbol, verificando el correcto funcionamiento de las rotaciones necesarias para restablecer el equilibrio. De esta manera, confirmamos que la estructura mantiene las propiedades características de un árbol AVL durante su funcionamiento.

Problema 3

Análisis y planteo

El problema consiste en determinar una forma eficiente de distribuir una noticia desde una aldea de origen hacia todas las demás aldeas disponibles, garantizando que cada una reciba el mensaje una única vez y minimizando la distancia total recorrida por las palomas.

A las aldeas podemos representarlas como vértices de un grafo, y las rutas entre ellas como aristas ponderadas por la distancia existente entre cada par de aldeas. Como el objetivo es conectar todas las aldeas minimizando la suma total de las distancias recorridas, el problema puede modelarse como la construcción de un Árbol de Expansión Mínima.

Las operaciones críticas son la representación de las conexiones entre aldeas, la exploración de vecinos y la selección iterativa de la conexión de menor costo que permita incorporar una nueva aldea a la red de comunicación. Para resolver este problema usamos una estructura de grafo ponderado junto con el algoritmo de Prim, el cual permite obtener un árbol de expansión mínima de manera eficiente.

Decidimos usar esta estructura y algoritmo porque el objetivo no consiste en encontrar el camino más corto hacia una única aldea, sino en construir una red completa de distribución minimizando el costo total de todas las conexiones utilizadas.

Implementación / Resolución

Para la solución, lo que hicimos fue utilizar un grafo representado por una colección de vértices. Cada vértice almacena el nombre de una aldea, sus aldeas vecinas y la distancia asociada a cada conexión.

Las rutas entre aldeas fueron cargadas desde el archivo de texto brindado por la cátedra, donde cada línea contiene una aldea de origen, una aldea de destino y la distancia entre ambas.

Para obtener la distribución óptima de la noticia hicimos uso del algoritmo de Prim. Inicialmente todas las aldeas se consideran no visitadas y con distancia infinita, excepto la aldea de origen Peligros, cuya distancia inicial es cero. Posteriormente usamos una cola de prioridad implementada sobre un montículo binario para seleccionar en cada paso la conexión de menor costo disponible.

A medida que el algoritmo avanza, cada aldea almacena un predecesor que indica desde qué aldea debe recibir la noticia. De esta forma, al finalizar la ejecución se obtiene un árbol de expansión mínima que permite reconstruir la red completa de comunicación.

Finalmente, el programa muestra lo solicitado en la consigna:

- La lista de aldeas ordenadas alfabéticamente.
- La aldea desde la cual cada una recibe la noticia.
- Las aldeas a las que debe reenviar el mensaje.

- La suma total de las distancias recorridas por todas las palomas utilizadas en la transmisión.

Resultados y pruebas

Se verificó el correcto funcionamiento del programa utilizando el archivo de datos provisto con las rutas entre las aldeas. Como resultado, se obtuvo una lista ordenada de todas las aldeas participantes y una estructura de comunicación que garantiza que cada aldea reciba la noticia una única vez.

Las pruebas realizadas permitieron comprobar que el algoritmo genera correctamente las relaciones de recepción y envío para cada aldea, respetando la estructura producida por el árbol de expansión mínima. Asimismo, se verificó que la distancia total calculada corresponde a la suma de las conexiones seleccionadas por el algoritmo.

Los resultados obtenidos muestran que la estrategia implementada permite minimizar la distancia total recorrida por las palomas mensajeras, cumpliendo con el objetivo planteado en el enunciado.

ANÁLISIS DE RESULTADOS

Durante el desarrollo del trabajo observamos la importancia de seleccionar estructuras de datos acordes a las operaciones predominantes de cada problema.

En el problema de la sala de emergencias, el uso de un montículo binario permitió administrar prioridades de manera eficiente, garantizando que los pacientes con mayor nivel de riesgo fueran atendidos antes que el resto. Se verificó que tanto la inserción de nuevos pacientes como la extracción del siguiente paciente a atender mantienen complejidad $O(\log n)$, lo que resulta adecuado para escenarios donde la cantidad de pacientes puede variar constantemente.

Por otro lado, en la base de datos de temperaturas, la utilización de un árbol AVL permitió mantener los datos ordenados y balanceados automáticamente. Gracias a esto, las operaciones de búsqueda, inserción y eliminación conservaron complejidad $O(\log n)$, independientemente del orden en que fueron incorporadas las mediciones. Asimismo, las consultas sobre rangos de fechas pudieron realizarse aprovechando el recorrido ordenado del árbol.

En el tercer problema, relacionado con la construcción de una red de comunicación entre aldeas mediante grafos, pudimos analizar la utilidad de las estructuras de datos no lineales para representar relaciones complejas entre múltiples elementos. La implementación del grafo permitió modelar las conexiones existentes entre las distintas aldeas, mientras que la aplicación del algoritmo de Prim nos permitió obtener un árbol de expansión mínima, seleccionando únicamente las conexiones necesarias para enlazar todas las aldeas con el menor costo total posible. Para optimizar el proceso de selección de vértices empleamos una cola de prioridad basada en un montículo binario, lo que permitió reducir el tiempo requerido para identificar la próxima conexión más conveniente. Gracias a esta combinación de estructuras y algoritmos, se obtuvo una solución eficiente tanto en términos de costo como de complejidad computacional.

USO ÉTICO DE INTELIGENCIA ARTIFICIAL

Herramientas utilizadas:

- Chat GPT, Gemini.

Descripción del uso:

- Asistencia para redactar explicaciones técnicas y mejorar la claridad del texto.
- Consulta teórica sobre complejidad computacional y uso de estructuras jerárquicas.
- Ayuda en depuración de errores sintácticos y validación lógica de algunas funciones.

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

TRABAJO EN EQUIPO

El presente trabajo práctico fue desarrollado de manera colaborativa entre los integrantes del grupo, distribuyendo las tareas de acuerdo con las distintas etapas del proyecto y las necesidades de cada ejercicio. Sin embargo, las decisiones de diseño, análisis e implementación fueron discutidas en conjunto, con el objetivo de que todos entendiéramos el funcionamiento de las soluciones desarrolladas.

La comunicación y coordinación del trabajo se llevó a cabo mediante WhatsApp y GitHub, utilizando un repositorio compartido para gestionar versiones, integrar cambios y mantener un seguimiento del avance del proyecto. Periódicamente se realizaron revisiones grupales del código con el fin de evaluar su funcionamiento, optimizar soluciones y resolver inconvenientes surgidos durante la implementación.

Entre las principales dificultades encontradas se destacaron la implementación de las operaciones de balanceo en los árboles AVL, el manejo de consultas sobre rangos de fechas en la base de datos de temperaturas, la utilización de colas de prioridad para la simulación de la sala de emergencias y la aplicación de algoritmos sobre grafos para determinar rutas eficientes entre aldeas. Estas dificultades fueron abordadas mediante análisis y pruebas adicionales sobre el código hasta alcanzar una solución funcional y acorde a los requisitos establecidos en la consigna.

CONCLUSIONES

El trabajo permitió afianzar conocimientos sobre estructuras jerárquicas, algoritmos de recorrido y búsqueda, balanceo de árboles, colas de prioridad y representación de grafos, comprendiendo su utilidad para resolver problemas reales de manera eficiente.

Durante el desarrollo de este trabajo observamos que la elección adecuada de una estructura de datos influye directamente en el rendimiento, la eficiencia y la escalabilidad de una aplicación. La utilización de grafos para representar conexiones entre aldeas y del algoritmo de Prim para obtener un árbol de expansión mínima permitió observar cómo es posible optimizar recursos y minimizar costos de conexión dentro de una red.

Asimismo, la incorporación de precondiciones, postcondiciones y manejo de excepciones nos permitió obtener una solución más robusta, confiable y fácil de mantener, garantizando la validación de los datos de entrada y el correcto funcionamiento de cada componente del sistema.

Otro aspecto importante fue la comprensión del impacto que tienen las estructuras auxiliares, como las colas de prioridad implementadas mediante montículos binarios, en la mejora de la complejidad temporal de los algoritmos. Esto permitió analizar no solo la corrección de la solución obtenida, sino también su eficiencia computacional frente a posibles escenarios de mayor tamaño.

BIBLIOGRAFÍA

Laboratorio de Informática y Computación Aplicada (LICA). (2026). *Precondiciones, Postcondiciones e Invariantes: de los límites de la computación a la especificación de algoritmos*. Facultad de Ingeniería, Universidad Nacional de Entre Ríos (UNER). <https://docs.google.com/document/d/1pO310W2b2Qj0Kq8uzstduWgkdRepy2GH4QSO8i1aXps/edit?tab=t.0>

Miller, B., & Ranum, D. (s.f.). *Solución de problemas con algoritmos y estructuras de datos usando Python*. Luther College. <https://runestone.academy/ns/books/published/pythoned/index.html>